KURA.

Sistema de Gestão Veterinária Digital

FIAP Challenge 2026 · Parceiro: Clyvo Vet
DevOps Tools & Cloud Computing · Turma 2TDS · 2026

Clayton Alves	RM562285
Felipe Ferrete Soares Lemes	RM562999
Guilherme Sola	RM563674
Gustavo Bosak	RM566315
Nikolas Brisola	RM564371

01 Descrição do Projeto

O KURA é um sistema de gestão veterinária digital desenvolvido para a Clyvo Vet, com foco em continuidade do cuidado e engajamento na jornada de saúde do pet. A plataforma resolve um problema central da veterinária moderna: a jornada do pet é episódica e reativa — o tutor só interage com a clínica em momentos de crise.

O KURA transforma isso em uma experiência contínua, preventiva e inteligente.

Microsserviços do Sistema

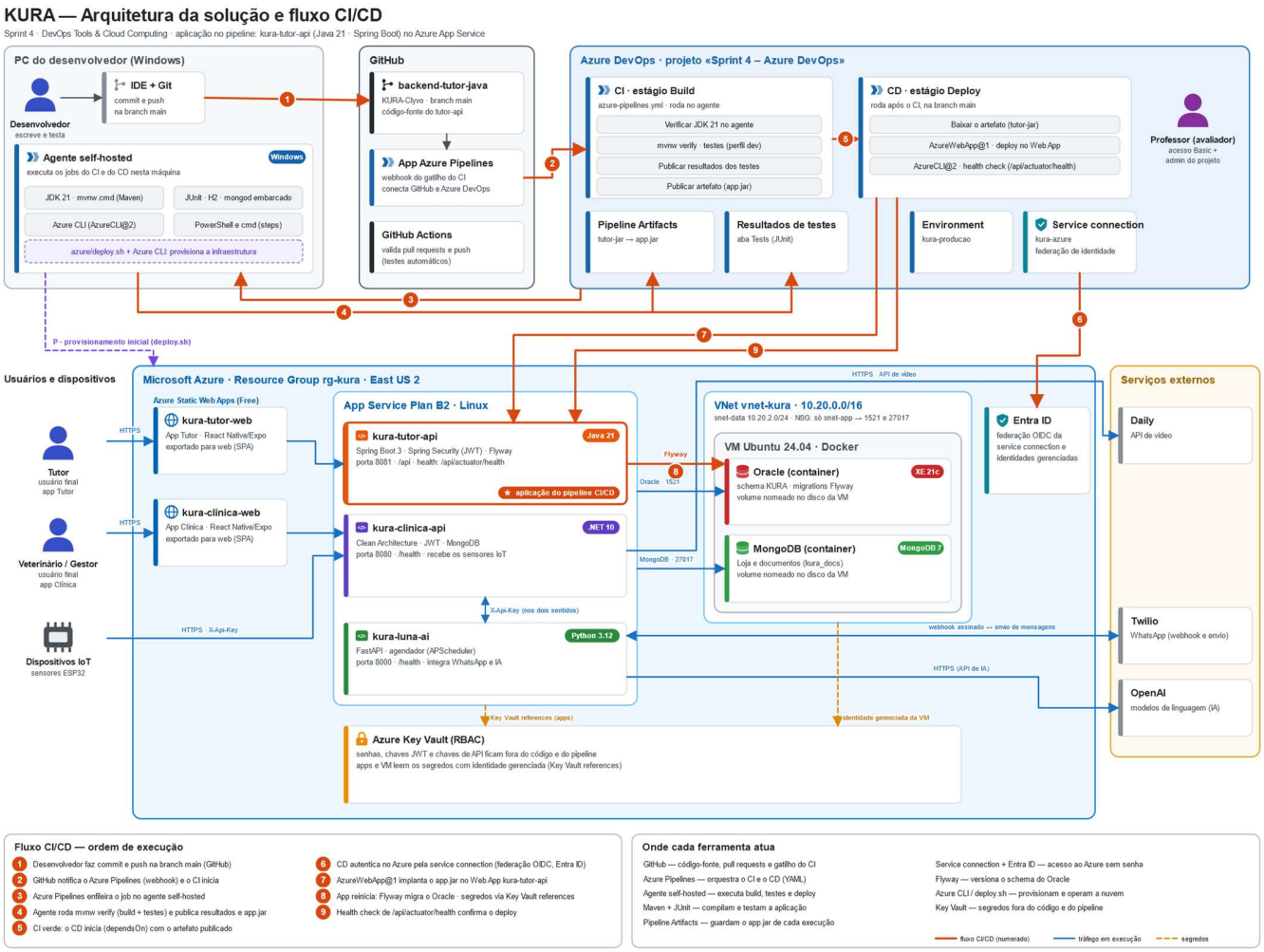
Serviço	Tecnologia	Porta	Descrição
kura-api	ASP.NET Core 10 · EF Core · Clean Architecture	8080	Backend clínico: vets, pets, eventos, IoT, dashboard
kura-tutor	Spring Boot 3.2 · Java 21 · Flyway · Caffeine	8081	Portal tutor: auth JWT, agendamentos, timeline, LGPD
luna-ai	FastAPI · Python 3.12 · YOLOv8n · MobileNetV3	8000	Triagem IA via WhatsApp (Twilio) + ID de raça por foto
oracle-db	Oracle XE 21c slim	1521 / 9092	Banco relacional compartilhado · volume persistente

02 Arquitetura do Sistema

Principais Mudanças em Relação à Arquitetura Anterior foi a migração de ACI para Azure App Service. As aplicações de backend (clinica-api, tutor-api, luna-ai) deixaram de ser executadas em Azure Container Instances individuais e passaram para Web Apps dedicados dentro de um App Service Plan Linux gerenciado. O modelo assegura terminação TLS nativa, suporte a Always-On sem hibernação de contêineres, integração bidirecional com VNet e fluxo de release simplificado via pacotes Zip e Jar. Também foi adiciona a persistência de dados em Máquina Virtual dedicada (VM Ubuntu): O Oracle XE 21c e o MongoDB 7 foram migrados do ACI, onde a persistência de volumes complexos de oradata apresentava limitações de montagem SMB, para containers Docker instanciados em uma VM dedicada alocada na sub-rede snet-data. A retenção de dados é realizada em volumes nomeados diretamente no SSD gerenciado da máquina virtual. Ainda foi implantada a gestão centralizada de segredos com Azure Key Vault e RBAC, na qual o cofre kv-kura-* centraliza o armazenamento de credenciais operacionais do Oracle e MongoDB, chaves criptográficas JWT, tokens de serviços terceiros (Twilio, OpenAI, Daily, credenciais IoT) e strings de conexão compostas. Os Web Apps consomem essas variáveis sensíveis através de Key Vault References atreladas a Managed Identities de sistema, dispensando configurações em texto legível nos repositórios ou painéis. Os frontends React Native por sua vez são exportados como SPAs estáticos e distribuídos globalmente pelo Azure Static Web Apps, provendo roteamento perimetral via CDN e regras de reescrita descritas em staticwebapp.config.json.

Além disso, o processo de deploy foi feito pensando em um script de automação unificado azure/deploy.sh implementa o ciclo de vida completo da plataforma através das seguintes fases operacionais:

- check: Executa a verificação dos binários e dependências locais do operador (az CLI, curl, python3, dotnet SDK 10, JDK 21, Node.js 22, CLI do Static Web Apps), integridade dos submódulos Git e autenticação ativa na assinatura do Azure.
- infra: Provisiona a estrutura base: Resource Group, VNet, Subnets com delegações de serviço, NSG com regras de filtragem estritas, Key Vault com RBAC, VM Ubuntu de dados (incluindo script cloud-init para inicialização automatizada do Docker, Oracle XE 21c e MongoDB 7), App Service Plan Linux, os 3 Web Apps e os 2 Static Web Apps.
- build: Compila os artefatos de release dos projetos a partir dos submódulos correspondentes, gerando os pacotes finais em artifacts/ (clinica.zip, tutor.jar e luna.zip).
- apps: Configura as App Settings e referências do Key Vault nos Web Apps, realizando a publicação respeitando a precedência arquitetural: tutor-api é inicializada em primeiro lugar para execução automatizada das migrations Flyway no Oracle, sucedida por clinica-api e luna-ai.
- settings: Permite a atualização atômica de variáveis de ambiente, configurações de CORS e flags operacionais nos Web Apps sem a necessidade de reconstrução ou re-deploy dos binários.
- web: Compila as interfaces React Native/Expo injetando dinamicamente as URLs HTTPS finais de produção geradas para as APIs e publica os bundles nos respectivos Static Web Apps via SWA CLI.
- urls: Varre os recursos provisionados e gera uma tabela com todos os endpoints públicos ativos (APIs de backend e portais web).
- seed: Efetua chamadas HTTP autenticadas contra os endpoints do sistema para injetar dados iniciais de demonstração (perfil da clínica modelo, tutores, pacientes veterinários, registros fotográficos e massa de testes da IA).
- all: Orquestra a execução ponta a ponta em cadeia não assistida (check -> infra -> build -> apps -> web).
- destroy: Deleta de forma controlada o Resource Group completo e efetua o purge definitivo do cofre no Key Vault para evitar conflitos de soft-delete.



03 Vídeo Demonstrativo

Demonstração completa da entrega DevOps & Cloud do KURA.

Link: https://canva.link/6kkayu0mybokj01